Database Design Document: KitchenFlow Restaurant Order & Kitchen Display System

Document ID: KF-DBD-001 · Version 0.1 (draft) · 29 October 2026 · Author: Rathi · Related: KF-PDD-001, KF-SRS-001, KF-SAD-001

1. Introduction

1.1 Purpose

This document specifies the PostgreSQL schema of KitchenFlow at DDL level: tables, columns, types, constraints, indexes, roles, the concurrency-critical SQL, volumes, seed data and migrations. It is document 4 of 7. It implements the data requirements in SRS section 9 (DR-01 to DR-09) and the concurrency and data architecture in SAD sections 6 and 8. The SQL in section 5 is the content of the baseline migration V1 in executable form.

1.2 Scope

One database, kitchenflow, with one schema, kf, holding 26 tables. Out of scope: an outlet identifier for multi-outlet operation (a scale-out change described in SAD 5.6), customer personal data (none is stored), and anything outside PostgreSQL.

1.3 Inputs

Source	What this document takes from it
SRS section 9 and DR-01 to DR-09	Entities, attributes, data rules, volume basis
SRS section 4 (business rules)	Rules that need a constraint: BR-01, BR-02, BR-03, BR-08, BR-12, BR-15
SAD 6.2 to 6.4	Mechanisms: compare-and-set, unique constraints, version columns, lock order
SAD 8.1 to 8.3	Roles, planned constraints and indexes (refined and confirmed here)
SAD 5.4	Database timeouts and pool settings

1.4 How to read this document

The SQL blocks run in the order shown. Tables marked (Should) belong to Should-priority features; they are included in the baseline because the baseline is replaced wholesale until the first demo build (section 11). Requirement IDs appear in comments so each constraint can be traced.

2. Design conventions and decisions

ID	Decision	Reason
DD-01	One schema, kf, owned by role kf_migrator. The application connects as kf_app, which has data rights only.	Least privilege: the application cannot change the schema or tamper with the audit log (FR-AUD-02).
DD-02	Primary keys are BIGINT. Batch-heavy tables (order_line, order_line_modifier, order_line_component, ticket, ticket_item, audit_event) take ids from sequences with INCREMENT 50. All other tables use identity columns.	Hibernate switches off insert batching for identity columns. A sequence with an allocation size of 50 keeps the batching that SAD 5.5 assumes.
DD-03	Names are snake_case. The orders table is plural because order is a reserved word; every other table is singular. Constraint and index prefixes: fk_, uq_, ck_ and ix_; primary keys keep PostgreSQL's default _pkey names.	Predictable names make error messages and query plans readable.
DD-04	Money is NUMERIC(12,2). Timestamps are TIMESTAMPTZ in UTC. The business day is a DATE. There are no floating-point columns.	DR-01, DR-02, BR-18. Checked automatically (section 11.4).
DD-05	Status and type columns are VARCHAR with a named CHECK constraint, not native ENUM types.	Adding a value is a one-line constraint change; ENUM changes are awkward in migrations.

ID	Decision	Reason
DD-06	Tables edited concurrently carry version BIGINT NOT NULL DEFAULT 0: orders, ticket, item_stock, menu_item and app_user. Native UPDATE statements must increment it and set updated_at.	Optimistic locking (FR-KDS-07) stays consistent between JPA and hand-written SQL.
DD-07	Order lines snapshot the item name, unit price, modifier total, GST rate and quantity. Ticket items snapshot the item name.	History must not change when the menu does (BR-02).
DD-08	Referential actions are the default NO ACTION; nothing cascades. Referenced menu items and users are deactivated, never deleted.	DR-05: an accidental delete cannot erase history.
DD-09	Deliberate denormalisation: ticket.business_day, ticket.rush and ticket_item.item_name.	The station queue and the preparation-time report then read one table. The order lock keeps ticket.rush in step with orders.rush (SAD 6.3).
DD-10	No business logic in the database. Two integrity mechanisms only: CHECK constraints, and one trigger that makes audit_event append-only.	Rules stay testable in Java; the trigger protects the one table that must never change.
DD-11	audit_event has no foreign keys.	The audit trail must survive changes to other tables and must never block a business transaction.
DD-12	Hibernate runs with ddl-auto set to validate. Flyway owns the schema.	One source of truth; drift fails at start-up.
DD-13	Foreign-key columns whose parent is never deleted (created_by, recorded_by, item_id on order lines) are not indexed unless a query needs them.	Saves write cost on the busiest tables.

3. Table map

The schema has 26 tables owned by eight modules (SAD section 4.1), grouped as in Figure 1. Foreign keys cross a module boundary only in the directions the dependency rules allow: order to menu, payment to order, kitchen to order, inventory to menu, and every module to station (config) and to user ids (auth). One change from SAD 4.1: station is owned by config, because menu, kitchen and auth all reference it. audit_event has no foreign keys (DD-11).

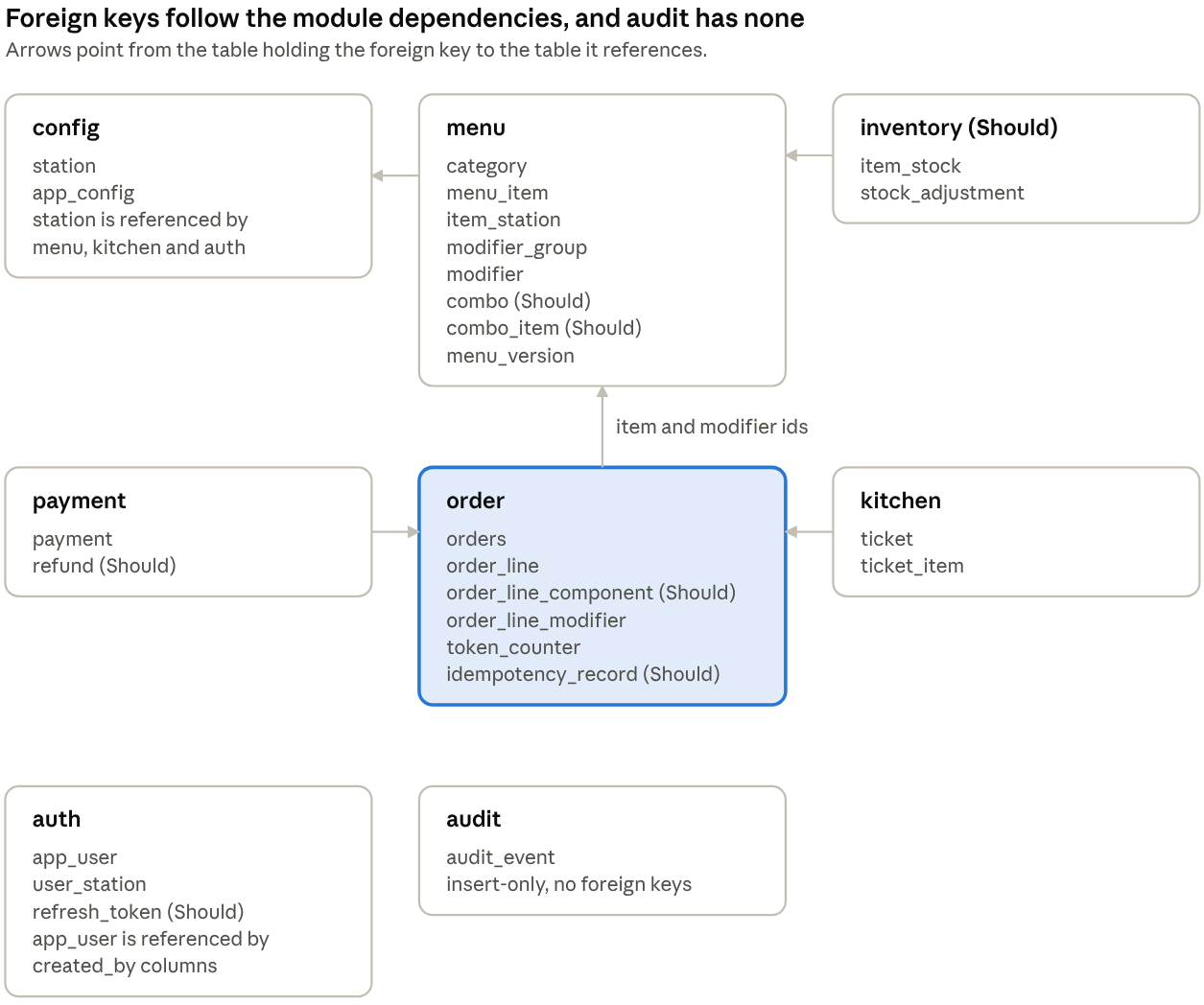


table map · 26 tables in 8 modules

Figure 1. Table map by module. Order is the hub; payment and kitchen point at it, and it points at the menu.

4. Roles, schema and session settings

Three things are created once, outside the migrations: the roles, the schema and the session limits of the application role (SAD 5.4 and 8.1). Flyway connects as kf_migrator; the application connects as kf_app.

```
-- Run once by the Compose init script as a superuser. Passwords arrive as
psql variables
-- taken from environment variables; the repository contains none.
CREATE ROLE kf_migrator LOGIN PASSWORD :'migrator_password';
CREATE ROLE kf_app      LOGIN PASSWORD :'app_password';
CREATE DATABASE kitchenflow OWNER kf_migrator;

-- Then, connected to the kitchenflow database:
CREATE SCHEMA kf AUTHORIZATION kf_migrator;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA kf TO kf_app;

ALTER DEFAULT PRIVILEGES FOR ROLE kf_migrator IN SCHEMA kf
  GRANT SELECT, INSERT, UPDATE ON TABLES TO kf_app;
ALTER DEFAULT PRIVILEGES FOR ROLE kf_migrator IN SCHEMA kf
  GRANT USAGE, SELECT ON SEQUENCES TO kf_app;

ALTER ROLE kf_app SET search_path = kf;
ALTER ROLE kf_app SET timezone = 'UTC';
ALTER ROLE kf_app SET statement_timeout = '5s';
ALTER ROLE kf_app SET lock_timeout = '2s';
ALTER ROLE kf_app SET idle_in_transaction_session_timeout = '30s';
```

- kf_app can read, insert and update every table. It receives DELETE only where section 5.11 grants it, and it has no schema-change rights.
- Role settings apply to new sessions. Report queries raise the limit for their own transaction with SET LOCAL statement_timeout = '10s'.
- Passwords come from environment variables at container start.

5. Schema DDL (baseline migration V1)

Run in this order. Comments carry requirement IDs.

5.1 Sequences

```
-- Batch-heavy tables: INCREMENT 50 matches Hibernate's allocation size of 50
(DD-02)
CREATE SEQUENCE kf.seq_order_line           INCREMENT 50;
CREATE SEQUENCE kf.seq_order_line_modifier  INCREMENT 50;
CREATE SEQUENCE kf.seq_order_line_component INCREMENT 50;
CREATE SEQUENCE kf.seq_ticket               INCREMENT 50;
CREATE SEQUENCE kf.seq_ticket_item          INCREMENT 50;
CREATE SEQUENCE kf.seq_audit_event          INCREMENT 50;
```

Seed and generator scripts must take ids from the same sequences with nextval, so their ids never collide with the ones Hibernate allocates.

5.2 Reference data: station

```
CREATE TABLE kf.station (
  id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name          VARCHAR(40) NOT NULL,
  display_order INT         NOT NULL DEFAULT 0,
  active        BOOLEAN     NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_station_name UNIQUE (name)
);
```

5.3 Authentication and users

```
CREATE TABLE kf.app_user (
  id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username        VARCHAR(50)  NOT NULL,
  password_hash   VARCHAR(100) NOT NULL,           -- BCrypt, FR-AUTH-02
  role            VARCHAR(10)  NOT NULL,
  active          BOOLEAN      NOT NULL DEFAULT TRUE,
  failed_attempts INT          NOT NULL DEFAULT 0, -- lockout state, FR-AUTH-
06
  last_failed_at  TIMESTAMPTZ,
  locked_until    TIMESTAMPTZ,
  created_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
  version         BIGINT       NOT NULL DEFAULT 0,
  CONSTRAINT ck_user_role     CHECK (role IN
('CASHIER','KITCHEN','MANAGER','ADMIN')),
  CONSTRAINT ck_user_attempts CHECK (failed_attempts >= 0)
```

```
);
CREATE UNIQUE INDEX uq_app_user_username ON kf.app_user (lower(username));

CREATE TABLE kf.user_station (
  user_id    BIGINT NOT NULL,
  station_id BIGINT NOT NULL,
  PRIMARY KEY (user_id, station_id),              -- FR-AUTH-04
  CONSTRAINT fk_us_user    FOREIGN KEY (user_id)    REFERENCES kf.app_user
(id),
  CONSTRAINT fk_us_station FOREIGN KEY (station_id) REFERENCES kf.station
(id)
);

CREATE TABLE kf.refresh_token (                    -- (Should) FR-AUTH-07
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     BIGINT      NOT NULL,
  family_id   UUID        NOT NULL,
  token_hash  CHAR(64)    NOT NULL,                -- SHA-256 hex, never the
token itself
  expires_at  TIMESTAMPTZ NOT NULL,
  consumed_at TIMESTAMPTZ,
  revoked_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT fk_rt_user FOREIGN KEY (user_id) REFERENCES kf.app_user (id),
  CONSTRAINT uq_rt_hash UNIQUE (token_hash)
);
CREATE INDEX ix_rt_user    ON kf.refresh_token (user_id);
CREATE INDEX ix_rt_family  ON kf.refresh_token (family_id);
CREATE INDEX ix_rt_expires ON kf.refresh_token (expires_at);   -- purge job
```

Usernames are unique case-insensitively through the functional index. Lockout state lives on the user row so it survives restarts. Refresh tokens are stored as hashes and grouped by family, so reuse of a consumed token can revoke the whole family.

5.4 Configuration

```
CREATE TABLE kf.app_config (
  config_key   VARCHAR(60)  PRIMARY KEY,
  config_value VARCHAR(200) NOT NULL,
  updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_by   BIGINT,
  CONSTRAINT fk_config_user FOREIGN KEY (updated_by) REFERENCES kf.app_user
(id)
);
```

Keys: recall_window_seconds, unpaid_timeout_minutes, business_day_cutoff, sla_amber_percent, sla_red_percent, board_ready_highlight_minutes. Defaults come from the seed script, not from the schema.

5.5 Menu

```
CREATE TABLE kf.category (
  id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name          VARCHAR(60)  NOT NULL,
  display_order INT          NOT NULL DEFAULT 0,
  gst_rate      NUMERIC(5,2) NOT NULL,
  active        BOOLEAN      NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_category_name UNIQUE (name),
  CONSTRAINT ck_category_gst  CHECK (gst_rate >= 0 AND gst_rate <= 28)   --
FR-MENU-01
);

CREATE TABLE kf.menu_item (
  id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_id         BIGINT        NOT NULL,
  name                VARCHAR(80)   NOT NULL,
  description         VARCHAR(300),
  base_price          NUMERIC(12,2) NOT NULL,
  target_prep_seconds INT           NOT NULL,
  active              BOOLEAN       NOT NULL DEFAULT TRUE,
  available           BOOLEAN       NOT NULL DEFAULT TRUE,   -- the 86 flag,
FR-MENU-04
  track_stock         BOOLEAN       NOT NULL DEFAULT FALSE,
  created_at          TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at          TIMESTAMPTZ   NOT NULL DEFAULT now(),
  version             BIGINT        NOT NULL DEFAULT 0,
```

```
  CONSTRAINT fk_item_category FOREIGN KEY (category_id) REFERENCES
kf.category (id),
  CONSTRAINT ck_item_price    CHECK (base_price > 0),         -- FR-MENU-02
  CONSTRAINT ck_item_prep     CHECK (target_prep_seconds > 0)
);
CREATE UNIQUE INDEX uq_menu_item_name     ON kf.menu_item (lower(name));
CREATE INDEX        ix_menu_item_category ON kf.menu_item (category_id) WHERE
active;

CREATE TABLE kf.item_station (                                -- BR-09
  item_id    BIGINT NOT NULL,
  station_id BIGINT NOT NULL,
  PRIMARY KEY (item_id, station_id),
  CONSTRAINT fk_is_item    FOREIGN KEY (item_id)    REFERENCES kf.menu_item
(id),
  CONSTRAINT fk_is_station FOREIGN KEY (station_id) REFERENCES kf.station
(id)
);
CREATE INDEX ix_item_station_station ON kf.item_station (station_id);

CREATE TABLE kf.modifier_group (
  id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  item_id        BIGINT      NOT NULL,
  name           VARCHAR(60) NOT NULL,
  selection_type VARCHAR(8)  NOT NULL,
  min_select     INT         NOT NULL DEFAULT 0,
  max_select     INT         NOT NULL DEFAULT 1,
  display_order  INT         NOT NULL DEFAULT 0,
  CONSTRAINT fk_mg_item   FOREIGN KEY (item_id) REFERENCES kf.menu_item (id),
  CONSTRAINT ck_mg_type   CHECK (selection_type IN ('SINGLE','MULTIPLE')),
  CONSTRAINT ck_mg_range  CHECK (min_select >= 0 AND max_select >=
GREATEST(min_select, 1)),
  CONSTRAINT ck_mg_single CHECK (selection_type <> 'SINGLE' OR max_select =
1)   -- FR-MENU-03
);
CREATE INDEX ix_mg_item ON kf.modifier_group (item_id);

CREATE TABLE kf.modifier (
  id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  group_id      BIGINT        NOT NULL,
  name          VARCHAR(60)   NOT NULL,
  price_delta   NUMERIC(12,2) NOT NULL DEFAULT 0,
  active        BOOLEAN       NOT NULL DEFAULT TRUE,
```

```
  display_order INT           NOT NULL DEFAULT 0,
  CONSTRAINT fk_modifier_group FOREIGN KEY (group_id) REFERENCES
kf.modifier_group (id),
  CONSTRAINT uq_modifier_name  UNIQUE (group_id, name)
);

CREATE TABLE kf.combo (                                    -- (Should) FR-
MENU-07
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_id BIGINT        NOT NULL,                      -- supplies the
GST rate
  name        VARCHAR(80)   NOT NULL,
  price       NUMERIC(12,2) NOT NULL,
  active      BOOLEAN       NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_combo_category FOREIGN KEY (category_id) REFERENCES
kf.category (id),
  CONSTRAINT uq_combo_name     UNIQUE (name),
  CONSTRAINT ck_combo_price    CHECK (price > 0)
);

CREATE TABLE kf.combo_item (                               -- (Should)
  combo_id BIGINT NOT NULL,
  item_id  BIGINT NOT NULL,
  quantity INT    NOT NULL DEFAULT 1,
  PRIMARY KEY (combo_id, item_id),
  CONSTRAINT fk_ci_combo FOREIGN KEY (combo_id) REFERENCES kf.combo (id),
  CONSTRAINT fk_ci_item  FOREIGN KEY (item_id)  REFERENCES kf.menu_item (id),
  CONSTRAINT ck_ci_qty   CHECK (quantity > 0)
);

CREATE TABLE kf.menu_version (                             -- feeds the
menu ETag (SAD 5.3)
  id      SMALLINT PRIMARY KEY DEFAULT 1,
  version BIGINT   NOT NULL DEFAULT 0,
  CONSTRAINT ck_menu_version_single CHECK (id = 1)
);
INSERT INTO kf.menu_version DEFAULT VALUES;
```

- ck_mg_range and ck_mg_single enforce FR-MENU-03: a single-choice group always allows exactly one selection.
- menu_version is a one-row counter incremented in the same transaction as every menu change; it becomes the ETag.

- BR-09 (every item has at least one station) is enforced by the service and checked by a seed test. A deferrable constraint trigger would make it a database rule; that is optional hardening (OI-03).

5.6 Inventory (Should)

```
CREATE TABLE kf.item_stock (
  item_id    BIGINT      PRIMARY KEY,
  quantity   INT         NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  version    BIGINT      NOT NULL DEFAULT 0,
  CONSTRAINT fk_stock_item FOREIGN KEY (item_id) REFERENCES kf.menu_item
(id),
  CONSTRAINT ck_stock_qty  CHECK (quantity >= 0)       -- NFR-CONC-02 safety
net
);

CREATE TABLE kf.stock_adjustment (
  id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  item_id    BIGINT       NOT NULL,
  delta      INT          NOT NULL,
  reason     VARCHAR(200) NOT NULL,                  -- FR-INV-05
  actor_id   BIGINT       NOT NULL,
  created_at TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CONSTRAINT fk_adj_item  FOREIGN KEY (item_id)  REFERENCES kf.menu_item
(id),
  CONSTRAINT fk_adj_actor FOREIGN KEY (actor_id) REFERENCES kf.app_user (id),
  CONSTRAINT ck_adj_delta CHECK (delta <> 0)
);
CREATE INDEX ix_adj_item ON kf.stock_adjustment (item_id, created_at DESC);
```

Stock decrements use the conditional UPDATE in section 8. ck_stock_qty is the safety net that makes overselling impossible even if application code is wrong.

5.7 Orders

```
CREATE TABLE kf.orders (
  id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_number      VARCHAR(24)   NOT NULL,           -- KF-YYYYMMDD-NNNN,
built by the application
  business_day      DATE          NOT NULL,           -- BR-16
  token             INT           NOT NULL,           -- BR-01
```

```
  order_type         VARCHAR(10)   NOT NULL,
  status             VARCHAR(12)   NOT NULL DEFAULT 'CREATED',
  rush               BOOLEAN       NOT NULL DEFAULT FALSE,
  note               VARCHAR(200),
  subtotal           NUMERIC(12,2) NOT NULL,
  tax_total          NUMERIC(12,2) NOT NULL,
  total              NUMERIC(12,2) NOT NULL,
  cancel_reason_code VARCHAR(30),
  cancel_reason_text VARCHAR(200),
  cancelled_by       BIGINT,
  created_by         BIGINT        NOT NULL,
  created_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
  paid_at            TIMESTAMPTZ,
  ready_at           TIMESTAMPTZ,
  completed_at       TIMESTAMPTZ,
  cancelled_at       TIMESTAMPTZ,
  version            BIGINT        NOT NULL DEFAULT 0,
  CONSTRAINT uq_orders_number    UNIQUE (order_number),
  CONSTRAINT uq_orders_day_token UNIQUE (business_day, token),
  CONSTRAINT fk_orders_creator   FOREIGN KEY (created_by)   REFERENCES
kf.app_user (id),
  CONSTRAINT fk_orders_canceller FOREIGN KEY (cancelled_by) REFERENCES
kf.app_user (id),
  CONSTRAINT ck_orders_type   CHECK (order_type IN
('DINE_IN','TAKEAWAY','DELIVERY')),
  CONSTRAINT ck_orders_status CHECK (status IN

('CREATED','PAID','IN_KITCHEN','READY','COMPLETED','CANCELLED')),
  CONSTRAINT ck_orders_token  CHECK (token > 0),
  CONSTRAINT ck_orders_money  CHECK (subtotal >= 0 AND tax_total >= 0
                                     AND total = subtotal + tax_total),   --
BR-02, BR-03
  CONSTRAINT ck_orders_paid   CHECK (status NOT IN
('PAID','IN_KITCHEN','READY','COMPLETED')
                                     OR paid_at IS NOT NULL),             --
BR-07
  CONSTRAINT ck_orders_done   CHECK (status <> 'COMPLETED' OR completed_at IS
NOT NULL),
  CONSTRAINT ck_orders_cancel CHECK ((status = 'CANCELLED') =
            (cancel_reason_code IS NOT NULL AND cancelled_at IS NOT NULL))
-- BR-05, FR-ORD-07
);
```

```
CREATE INDEX ix_orders_status_created ON kf.orders (status, created_at DESC);
CREATE INDEX ix_orders_day_status     ON kf.orders (business_day, status);
CREATE INDEX ix_orders_open   ON kf.orders (created_at DESC)
  WHERE status IN ('CREATED','IN_KITCHEN','READY');
CREATE INDEX ix_orders_unpaid ON kf.orders (created_at) WHERE status =
'CREATED';  -- FR-ORD-12

CREATE TABLE kf.order_line (
  id             BIGINT PRIMARY KEY DEFAULT nextval('kf.seq_order_line'),
  order_id       BIGINT        NOT NULL,
  line_no        INT           NOT NULL,
  item_id        BIGINT,
  combo_id       BIGINT,
  item_name      VARCHAR(80)   NOT NULL,               -- snapshots, BR-02
  unit_price     NUMERIC(12,2) NOT NULL,
  modifier_total NUMERIC(12,2) NOT NULL DEFAULT 0,
  gst_rate       NUMERIC(5,2)  NOT NULL,
  quantity       INT           NOT NULL,
  line_subtotal  NUMERIC(12,2) NOT NULL,
  line_tax       NUMERIC(12,2) NOT NULL,
  line_total     NUMERIC(12,2) NOT NULL,
  CONSTRAINT fk_ol_order  FOREIGN KEY (order_id) REFERENCES kf.orders (id),
  CONSTRAINT fk_ol_item   FOREIGN KEY (item_id)  REFERENCES kf.menu_item
(id),
  CONSTRAINT fk_ol_combo  FOREIGN KEY (combo_id) REFERENCES kf.combo (id),
  CONSTRAINT uq_ol_order_line UNIQUE (order_id, line_no),
  CONSTRAINT ck_ol_target CHECK ((item_id IS NULL) <> (combo_id IS NULL)),
  CONSTRAINT ck_ol_qty    CHECK (quantity BETWEEN 1 AND 50),            --
FR-ORD-01
  CONSTRAINT ck_ol_price  CHECK (unit_price >= 0 AND unit_price +
modifier_total >= 0),
  CONSTRAINT ck_ol_sub    CHECK (line_subtotal = (unit_price +
modifier_total) * quantity),
  CONSTRAINT ck_ol_tax    CHECK (line_tax = round(line_subtotal * gst_rate /
100, 2)),  -- BR-03
  CONSTRAINT ck_ol_total  CHECK (line_total = line_subtotal + line_tax)
);

CREATE TABLE kf.order_line_component (                       -- (Should)
combo expansion snapshot
  id            BIGINT PRIMARY KEY DEFAULT
nextval('kf.seq_order_line_component'),
  order_line_id BIGINT      NOT NULL,
```

```
  item_id       BIGINT      NOT NULL,
  item_name     VARCHAR(80) NOT NULL,
  quantity      INT         NOT NULL,
  CONSTRAINT fk_olc_line FOREIGN KEY (order_line_id) REFERENCES kf.order_line
(id),
  CONSTRAINT fk_olc_item FOREIGN KEY (item_id)       REFERENCES kf.menu_item
(id),
  CONSTRAINT ck_olc_qty  CHECK (quantity > 0)
);
CREATE INDEX ix_olc_line ON kf.order_line_component (order_line_id);

CREATE TABLE kf.order_line_modifier (
  id            BIGINT PRIMARY KEY DEFAULT
nextval('kf.seq_order_line_modifier'),
  order_line_id BIGINT        NOT NULL,
  modifier_id   BIGINT,
  modifier_name VARCHAR(60)   NOT NULL,                  -- snapshot
  price_delta   NUMERIC(12,2) NOT NULL DEFAULT 0,
  CONSTRAINT fk_olm_line     FOREIGN KEY (order_line_id) REFERENCES
kf.order_line (id),
  CONSTRAINT fk_olm_modifier FOREIGN KEY (modifier_id)   REFERENCES
kf.modifier (id)
);
CREATE INDEX ix_olm_line ON kf.order_line_modifier (order_line_id);

CREATE TABLE kf.token_counter (
  business_day DATE PRIMARY KEY,
  last_token   INT  NOT NULL DEFAULT 0
);

CREATE TABLE kf.idempotency_record (                        -- (Should) FR-
ORD-10, BR-15
  id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id         BIGINT      NOT NULL,
  idem_key        VARCHAR(80) NOT NULL,
  request_hash    CHAR(64)    NOT NULL,
  response_status SMALLINT,
  response_body   JSONB,
  order_id        BIGINT,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT fk_idem_user     FOREIGN KEY (user_id)  REFERENCES kf.app_user
(id),
  CONSTRAINT fk_idem_order    FOREIGN KEY (order_id) REFERENCES kf.orders
```

```
(id),
  CONSTRAINT uq_idem_user_key UNIQUE (user_id, idem_key)
);
CREATE INDEX ix_idem_created ON kf.idempotency_record (created_at);   -- 24-
hour purge
```

- The cross-row totals (orders.subtotal equals the sum of line subtotals, and likewise for tax) cannot be a CHECK. They are verified by the consistency queries in section 14.
- ck_ol_sub, ck_ol_tax and ck_ol_total make the pricing arithmetic of BR-03 a database rule: a wrong total cannot be stored. round(numeric, 2) rounds half away from zero, which is half-up for the non-negative amounts used here.
- idempotency_record has no status column. The first request inserts the row and stores its response inside its own transaction, so no other session can see an unfinished row (SAD 6.2).
- Combo lines hold combo_id and no item_id; order_line_component records the combo's expansion at order time so routing does not depend on the current combo definition.

5.8 Payments

```
CREATE TABLE kf.payment (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id    BIGINT        NOT NULL,
  method      VARCHAR(4)    NOT NULL,
  amount      NUMERIC(12,2) NOT NULL,
  reference   VARCHAR(64),
  tendered    NUMERIC(12,2),                          -- FR-PAY-06
  change_due  NUMERIC(12,2),
  status      VARCHAR(14)   NOT NULL DEFAULT 'RECORDED',
  recorded_by BIGINT        NOT NULL,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uq_payment_order  UNIQUE (order_id),      -- FR-PAY-03
  CONSTRAINT fk_payment_order  FOREIGN KEY (order_id)    REFERENCES kf.orders
(id),
  CONSTRAINT fk_payment_user   FOREIGN KEY (recorded_by) REFERENCES
kf.app_user (id),
  CONSTRAINT ck_payment_method CHECK (method IN ('CASH','UPI','CARD')),
  CONSTRAINT ck_payment_amount CHECK (amount > 0),
  CONSTRAINT ck_payment_status CHECK (status IN
('RECORDED','REFUND_PENDING','REFUNDED')),
  CONSTRAINT ck_payment_cash   CHECK (tendered IS NULL OR
```

```
             (method = 'CASH' AND tendered >= amount AND change_due =
tendered - amount))
);
CREATE INDEX ix_payment_created ON kf.payment (created_at);

CREATE TABLE kf.refund (                                    -- (Should) FR-
PAY-07
  id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payment_id BIGINT        NOT NULL,
  amount     NUMERIC(12,2) NOT NULL,
  status     VARCHAR(14)   NOT NULL DEFAULT 'REFUND_PENDING',
  reason     VARCHAR(200)  NOT NULL,
  created_by BIGINT        NOT NULL,
  created_at TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT fk_refund_payment FOREIGN KEY (payment_id) REFERENCES kf.payment
(id),
  CONSTRAINT fk_refund_user    FOREIGN KEY (created_by) REFERENCES
kf.app_user (id),
  CONSTRAINT ck_refund_amount  CHECK (amount > 0),
  CONSTRAINT ck_refund_status  CHECK (status IN
('REFUND_PENDING','REFUNDED'))
);
CREATE UNIQUE INDEX uq_refund_payment_once ON kf.refund (payment_id);
```

The rule that the amount equals the order total (BR-08) is checked by the payment service inside the transaction, because it compares two tables. payment.status mirrors the refund state and is updated in the same transaction as the refund row.

5.9 Kitchen

```
CREATE TABLE kf.ticket (
  id             BIGINT PRIMARY KEY DEFAULT nextval('kf.seq_ticket'),
  order_id       BIGINT      NOT NULL,
  station_id     BIGINT      NOT NULL,
  business_day   DATE        NOT NULL,                -- denormalised, DD-09
  status         VARCHAR(12) NOT NULL DEFAULT 'NEW',
  rush           BOOLEAN     NOT NULL DEFAULT FALSE,  -- mirrors
orders.rush, DD-09
  target_seconds INT         NOT NULL,                -- BR-11
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at     TIMESTAMPTZ,
  done_at        TIMESTAMPTZ,
```

```
  cancelled_at   TIMESTAMPTZ,
  cancel_ack_at  TIMESTAMPTZ,                          -- FR-KDS-08
  recall_count   INT         NOT NULL DEFAULT 0,
  version        BIGINT      NOT NULL DEFAULT 0,       -- FR-KDS-07
  CONSTRAINT fk_ticket_order   FOREIGN KEY (order_id)   REFERENCES kf.orders
(id),
  CONSTRAINT fk_ticket_station FOREIGN KEY (station_id) REFERENCES kf.station
(id),
  CONSTRAINT uq_ticket_order_station UNIQUE (order_id, station_id),
  CONSTRAINT ck_ticket_status  CHECK (status IN
('NEW','IN_PROGRESS','DONE','CANCELLED')),
  CONSTRAINT ck_ticket_target  CHECK (target_seconds > 0),
  CONSTRAINT ck_ticket_started CHECK (status IN ('NEW','CANCELLED') OR
started_at IS NOT NULL),
  CONSTRAINT ck_ticket_done    CHECK ((status = 'DONE') = (done_at IS NOT
NULL)),
  CONSTRAINT ck_ticket_cancel  CHECK ((status = 'CANCELLED') = (cancelled_at
IS NOT NULL))
);
CREATE INDEX ix_ticket_station_open ON kf.ticket (station_id, rush DESC,
created_at)
  WHERE status IN ('NEW','IN_PROGRESS');                              --
station queue, BR-10
CREATE INDEX ix_ticket_day_station ON kf.ticket (business_day, station_id)
  WHERE status = 'DONE';                                              --
FR-RPT-03
CREATE INDEX ix_ticket_unacked ON kf.ticket (station_id)
  WHERE status = 'CANCELLED' AND cancel_ack_at IS NULL;               --
FR-KDS-08

CREATE TABLE kf.ticket_item (
  id            BIGINT PRIMARY KEY DEFAULT nextval('kf.seq_ticket_item'),
  ticket_id     BIGINT      NOT NULL,
  order_line_id BIGINT      NOT NULL,
  item_id       BIGINT      NOT NULL,
  item_name     VARCHAR(80) NOT NULL,                 -- snapshot for the
screen, DD-09
  quantity      INT         NOT NULL,
  CONSTRAINT fk_tki_ticket FOREIGN KEY (ticket_id)     REFERENCES kf.ticket
(id),
  CONSTRAINT fk_tki_line   FOREIGN KEY (order_line_id) REFERENCES
kf.order_line (id),
  CONSTRAINT fk_tki_item   FOREIGN KEY (item_id)       REFERENCES
```

```
kf.menu_item (id),
  CONSTRAINT ck_tki_qty    CHECK (quantity > 0)
);
CREATE INDEX ix_tki_ticket ON kf.ticket_item (ticket_id);
```

A recall (DONE back to IN_PROGRESS) must clear done_at in the same statement; ck_ticket_done rejects the update otherwise. uq_ticket_order_station also serves lookups of a single order's tickets.

5.10 Audit

```
CREATE TABLE kf.audit_event (
  id             BIGINT PRIMARY KEY DEFAULT nextval('kf.seq_audit_event'),
  occurred_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  actor_id       BIGINT,                              -- no foreign key, DD-
11
  actor_role     VARCHAR(10)  NOT NULL,               -- includes SYSTEM
  entity_type    VARCHAR(30)  NOT NULL,
  entity_id      BIGINT       NOT NULL,
  action         VARCHAR(40)  NOT NULL,
  from_state     VARCHAR(20),
  to_state       VARCHAR(20),
  reason         VARCHAR(200),
  correlation_id VARCHAR(64),
  details        JSONB
);
CREATE INDEX ix_audit_entity ON kf.audit_event (entity_type, entity_id,
occurred_at DESC);
CREATE INDEX ix_audit_time   ON kf.audit_event (occurred_at DESC);
CREATE INDEX ix_audit_actor  ON kf.audit_event (actor_id, occurred_at DESC);

CREATE FUNCTION kf.reject_audit_change() RETURNS trigger LANGUAGE plpgsql AS
$$
BEGIN
  RAISE EXCEPTION 'audit_event is append-only';
END $$;

CREATE TRIGGER trg_audit_append_only
  BEFORE UPDATE OR DELETE ON kf.audit_event
  FOR EACH ROW EXECUTE FUNCTION kf.reject_audit_change();     -- FR-AUD-02
```

5.11 Storage parameters and grants

```
-- Frequently updated rows: vacuum earlier; leave room so updates stay on the
same page
ALTER TABLE kf.ticket        SET (autovacuum_vacuum_scale_factor = 0.05);
ALTER TABLE kf.orders        SET (autovacuum_vacuum_scale_factor = 0.05);
ALTER TABLE kf.item_stock    SET (fillfactor = 70);
ALTER TABLE kf.token_counter SET (fillfactor = 50);

-- Least privilege: DELETE only where the application purges rows or edits
link rows (DR-09)
GRANT DELETE ON kf.item_station, kf.user_station, kf.combo_item,
                kf.idempotency_record, kf.refresh_token TO kf_app;

-- The application may only insert into and read the audit log (FR-AUD-02)
REVOKE UPDATE ON kf.audit_event FROM kf_app;
```

The fill-factor settings apply to item_stock and token_counter because their updated columns are not indexed, so updates can stay on the same page. ticket and orders update indexed status columns, so they gain from earlier vacuuming instead.

6. Integrity matrix

Rule	Database mechanism	Application mechanism
BR-01 Unique token per business day	UNIQUE (business_day, token); token_counter upsert (S1)	TokenService
BR-02, BR-03 Money, snapshots, GST	NUMERIC(12,2); ck_orders_money; ck_ol_sub, ck_ol_tax, ck_ol_total	PricingService
BR-04 Unavailable item rejected	None (depends on current state)	OrderService reads current availability in the transaction
BR-05 Cancellation rules and reason	ck_orders_cancel	Role and state checks
BR-06 READY when all tickets DONE	None	Order lock, then count open tickets (S4, S7, S8)

Rule	Database mechanism	Application mechanism
BR-07, BR-08 Pay first, amount equals total	ck_orders_paid; ck_payment_amount	PaymentService compares amount with total
BR-09 Every item has a station	Primary key (item_id, station_id)	Service check and seed test
BR-12 Stock never negative	ck_stock_qty	Conditional UPDATE (S3)
BR-13 Audit with every change	None	Same transaction as the change
BR-15 Idempotency	UNIQUE (user_id, idem_key)	IdempotencyService (S2)
BR-16, BR-18 Business day and UTC	DATE and TIMESTAMPTZ columns	Clock and configuration
FR-PAY-03 One payment per order	uq_payment_order	Compare-and-set (S5)
FR-KDS-07 One writer per ticket	version column	JPA @Version
FR-AUD-02 Append-only audit	No UPDATE or DELETE grant; trigger	None
DR-03 Foreign keys and checks	All FK and CK constraints in section 5	Verified by tests (section 14)
DR-05 Never delete referenced rows	NO ACTION foreign keys; DELETE not granted on core tables	Soft deactivation

7. Indexes and access paths

7.1 Index catalogue (the indexes that matter)

Index	Table	Definition	Serves
uq_orders_number, uq_orders_day_token	orders	Unique	Lookup by order number or token; BR-01
ix_orders_status_created	orders	(status, created_at DESC)	Order search by status, newest first (FR-ORD-09)
ix_orders_day_status	orders	(business_day, status)	Day filters and reports
ix_orders_open	orders	(created_at DESC), partial: open statuses	Manager board
ix_orders_unpaid	orders	(created_at), partial: CREATED	Unpaid timeout job (FR-ORD-12)
uq_ol_order_line	order_line	(order_id, line_no)	Loading the lines of one order
uq_ticket_order_station	ticket	(order_id, station_id)	Tickets of an order; READY check
ix_ticket_station_open	ticket	(station_id, rush DESC, created_at), partial: NEW and IN_PROGRESS	Station queue in display order, with no sort step
ix_ticket_day_station	ticket	(business_day, station_id), partial: DONE	Preparation-time report

Index	Table	Definition	Serves
ix_ticket_unacked	ticket	(station_id), partial: cancelled and unacknowledged	Cancelled notices
uq_payment_order	payment	(order_id)	One payment per order; join for reports
ix_audit_entity	audit_event	(entity_type, entity_id, occurred_at DESC)	History of one entity
ix_audit_time	audit_event	(occurred_at DESC)	Audit search by date
ix_audit_actor	audit_event	(actor_id, occurred_at DESC)	Audit search by actor
uq_idem_user_key	idempotency_record	(user_id, idem_key)	Duplicate detection; BR-15
uq_app_user_username	app_user	(lower(username))	Login lookup

Partial indexes keep the hottest indexes tiny: the station queue index holds only open tickets, which number in the tens, however large the history grows.

7.2 Plan targets

To be checked with EXPLAIN (ANALYZE, BUFFERS) on the 100,000-order dataset after ANALYZE, and asserted by tests through EXPLAIN (FORMAT JSON).

Statement	Expected plan	Fails the check if
S6 Station queue	Index scan on ix_ticket_station_open; no Sort node	Sequential scan or a Sort over ticket
S7 Open tickets of an order	Index scan on uq_ticket_order_station	Sequential scan on ticket
Order by token	Index scan on uq_orders_day_token	Sequential scan on orders

Statement	Expected plan	Fails the check if
Order search, one status, newest page	Index scan on ix_orders_status_created with LIMIT applied early	A Sort over all matching rows
Open orders (manager board)	Index scan on ix_orders_open	Sequential scan on orders
S10 Sales summary, 31 days	Range scan on ix_orders_day_status, then lookup on uq_payment_order	Elapsed time above 2 s
S12 Preparation times, 31 days	Range scan on ix_ticket_day_station	Elapsed time above 2 s
Audit by entity	Index scan on ix_audit_entity	Sequential scan on audit_event
Audit by date range	Range scan on ix_audit_time	Elapsed time above 300 ms

Plan output for each statement is saved in the repository after the first tuning pass, so a later regression shows up as a diff.

8. Critical SQL statements

Parameters are written as :name. S1 to S9 are the concurrency-critical statements from SAD section 6; S10 to S13 are the reports (FR-RPT-01 to 04). All run as kf_app.

```
-- S1 Token allocation: one atomic statement; the row lock is held until
commit (SAD 6.2)
INSERT INTO kf.token_counter AS c (business_day, last_token)
VALUES (:day, 1)
ON CONFLICT (business_day) DO UPDATE SET last_token = c.last_token + 1
RETURNING c.last_token;

-- S2 Idempotency, insert first (BR-15)
INSERT INTO kf.idempotency_record (user_id, idem_key, request_hash)
VALUES (:user, :key, :hash)
ON CONFLICT (user_id, idem_key) DO NOTHING;
-- 1 row inserted: first request, continue.
-- 0 rows: a duplicate that waited for any in-flight first request to commit.
Read the stored row:
```

```
SELECT request_hash, response_status, response_body
FROM kf.idempotency_record WHERE user_id = :user AND idem_key = :key;
-- same hash: replay the stored response; different hash: 422
IDEMPOTENCY_KEY_REUSED
-- before commit the first request stores its response:
UPDATE kf.idempotency_record
SET response_status = :status, response_body = :body, order_id = :order_id
WHERE user_id = :user AND idem_key = :key;

-- S3 Stock reservation, ascending item id, one statement per tracked item
(FR-INV-02)
UPDATE kf.item_stock
SET quantity = quantity - :qty, version = version + 1, updated_at = :now
WHERE item_id = :item AND quantity >= :qty;
-- 0 rows updated: out of stock, roll back the whole transaction (409
OUT_OF_STOCK)

-- S4 Order lock: first statement of bump, recall and cancel (SAD 6.3)
SELECT id, status, version FROM kf.orders WHERE id = :id FOR UPDATE;

-- S5 Payment compare-and-set (FR-PAY-03)
UPDATE kf.orders
SET status = 'PAID', paid_at = :now, updated_at = :now, version = version + 1
WHERE id = :id AND status = 'CREATED';
-- 0 rows updated: 409 (already paid, cancelled or unknown)

-- S6 Station queue (FR-KDS-02, BR-10)
SELECT t.id, t.order_id, t.status, t.rush, t.created_at, t.target_seconds,
t.version
FROM kf.ticket t
WHERE t.station_id = :station AND t.status IN ('NEW','IN_PROGRESS')
ORDER BY t.rush DESC, t.created_at ASC;

-- S7 Open tickets of an order (READY check, FR-KDS-06)
SELECT count(*) FROM kf.ticket
WHERE order_id = :id AND status IN ('NEW','IN_PROGRESS');

-- S8 READY compare-and-set, run only when S7 returns 0 and the order lock is
held
UPDATE kf.orders
SET status = 'READY', ready_at = :now, updated_at = :now, version = version +
1
WHERE id = :id AND status = 'IN_KITCHEN';
```

```
-- S9 Unpaid-order timeout: claim a batch without blocking on locked rows
(FR-ORD-12)
SELECT id FROM kf.orders
WHERE status = 'CREATED' AND created_at < :cutoff
ORDER BY created_at
LIMIT 50
FOR UPDATE SKIP LOCKED;

-- S10 Sales summary (FR-RPT-01): paid, non-cancelled orders
SELECT o.business_day, p.method, o.order_type,
       count(*)         AS orders,
       sum(o.subtotal)  AS subtotal,
       sum(o.tax_total) AS gst,
       sum(o.total)     AS total
FROM kf.orders o
JOIN kf.payment p ON p.order_id = o.id
WHERE o.business_day BETWEEN :from AND :to AND o.status <> 'CANCELLED'
GROUP BY o.business_day, p.method, o.order_type
ORDER BY o.business_day, p.method, o.order_type;

-- S11 Top items by quantity (FR-RPT-02); combo lines have no item_id and are
listed separately
SELECT l.item_id, l.item_name, sum(l.quantity) AS quantity, sum(l.line_total)
AS revenue
FROM kf.order_line l
JOIN kf.orders o ON o.id = l.order_id
WHERE o.business_day BETWEEN :from AND :to
  AND o.status IN ('IN_KITCHEN','READY','COMPLETED')
  AND l.item_id IS NOT NULL
GROUP BY l.item_id, l.item_name
ORDER BY quantity DESC, l.item_name
LIMIT :n;

-- S12 Kitchen cycle time per station, from ticket creation to done (FR-RPT-
03)
SELECT t.station_id,
       count(*) AS tickets,
       avg(extract(epoch FROM t.done_at - t.created_at)) AS avg_seconds,
       percentile_cont(0.5) WITHIN GROUP
         (ORDER BY extract(epoch FROM t.done_at - t.created_at)) AS
median_seconds,
       percentile_cont(0.9) WITHIN GROUP
```

```
         (ORDER BY extract(epoch FROM t.done_at - t.created_at)) AS
p90_seconds,
       avg((extract(epoch FROM t.done_at - t.created_at) >
t.target_seconds)::int) AS late_share
FROM kf.ticket t
WHERE t.business_day BETWEEN :from AND :to AND t.status = 'DONE'
GROUP BY t.station_id;

-- S13 Orders per hour of day (FR-RPT-04)
SELECT extract(hour FROM o.created_at AT TIME ZONE 'Asia/Kolkata')::int AS
hour,
       count(*) AS orders
FROM kf.orders o
WHERE o.business_day BETWEEN :from AND :to
GROUP BY 1
ORDER BY 1;
```

Notes: S10 to S13 run inside a read-only transaction with SET LOCAL statement_timeout = '10s' and behind the report bulkhead (SAD 5.4). S12 measures from ticket creation, which is when the customer's wait for that station starts; started_at is kept so a cook-time view can be added later.

9. Volume and capacity estimates

Based on the SRS design basis of 1,500 orders per day and about 8 audit records per order (DR-08). Sizes are estimates that assume average row widths of 0.12 to 0.35 KB including indexes; compare them with the generated dataset (section 10.9) and update this table.

Table	Rows per day	Rows after 12 months	Approximate size
orders	1,500	547,500	190 MB
order_line	3,750	1.37 million	340 MB
order_line_modifier	1,500	547,500	65 MB
ticket	3,000	1.1 million	275 MB
ticket_item	4,500	1.6 million	245 MB
payment	1,500	547,500	110 MB

Table	Rows per day	Rows after 12 months	Approximate size
audit_event	12,000	4.4 million	1.3 GB
idempotency_record	1,500 (purged after 24 hours)	About 3,000 live	Under 1 MB
Menu, users, configuration and the rest	Negligible	Under 1,000 rows each	Under 1 MB

Total: about 2.5 GB per year, comfortable on one small instance. The 100,000-order test dataset should occupy roughly 0.5 GB. Orders and audit records are kept for at least 12 months (DR-09); beyond that, monthly partitioning of audit_event is the first scale-out step (OI-04).

10. Seed data and test data

10.1 Principles

Seed data is deterministic, refuses to run on a non-empty database, and loads in under 60 seconds (FR-CFG-02). The catalogue, stations, configuration and stock are loaded by seed_demo.sql. Users are created by the application's seed runner through its own BCrypt password encoder, with the password taken from the SEED_PASSWORD environment variable, so no password or hash is stored in the repository. Prices and GST rates below are demonstration values, not tax advice.

10.2 Stations, users and configuration

Station	Display order
Fry	1
Grill	2
Assembly	3
Beverage	4

Username	Role	Stations
admin1	ADMIN	None
manager1	MANAGER	All (no binding needed)
cashier1, cashier2	CASHIER	None
fry1	KITCHEN	Fry
grill1	KITCHEN	Grill
assembly1	KITCHEN	Assembly
beverage1	KITCHEN	Beverage
multi1	KITCHEN	Fry and Grill (tests FR-AUTH-04 with two bindings)

Config key	Default
recall_window_seconds	600
unpaid_timeout_minutes	15
business_day_cutoff	04:00
sla_amber_percent	80
sla_red_percent	100
board_ready_highlight_minutes	5

10.3 Categories

Category	GST rate (demo)
Burgers	5
Chicken	5
Sides	5
Beverages	18
Desserts	5

10.4 Menu items (30)

Item	Category	Price (INR)	Stations	Target seconds	Stock
Classic Veg Burger	Burgers	99	Grill, Assembly	240	Not tracked
Cheese Veg Burger	Burgers	119	Grill, Assembly	240	Not tracked
Crispy Chicken Burger	Burgers	149	Fry, Assembly	300	Not tracked
Spicy Chicken Burger	Burgers	169	Fry, Assembly	300	Not tracked
Double Patty Burger	Burgers	199	Grill, Assembly	330	Not tracked
Paneer Wrap	Burgers	129	Grill, Assembly	270	Not tracked
Fried Chicken 2 pc	Chicken	179	Fry	420	Not tracked
Fried Chicken 4 pc	Chicken	329	Fry	480	50
Hot Wings 6 pc	Chicken	149	Fry	360	50
Popcorn Chicken	Chicken	139	Fry	300	Not tracked
Chicken Strips 4 pc	Chicken	189	Fry	360	Not tracked
Boneless Bites	Chicken	159	Fry	330	Not tracked
Regular Fries	Sides	79	Fry	180	Not tracked
Peri Peri Fries	Sides	99	Fry	180	Not tracked
Onion Rings	Sides	89	Fry	210	Not tracked
Veg Nuggets 6 pc	Sides	119	Fry	240	Not tracked
Coleslaw	Sides	49	Assembly	60	Not tracked
Garlic Bread	Sides	69	Grill	150	Not tracked
Cheese Dip	Sides	25	Assembly	20	Not tracked

Item	Category	Price (INR)	Stations	Target seconds	Stock
Cola	Beverages	69	Beverage	45	Not tracked
Lemon Iced Tea	Beverages	89	Beverage	60	Not tracked
Cold Coffee	Beverages	119	Beverage	120	Not tracked
Mango Shake	Beverages	129	Beverage	120	Not tracked
Bottled Water	Beverages	29	Beverage	20	Not tracked
Hot Coffee	Beverages	79	Beverage	90	Not tracked
Choco Lava Cake	Desserts	89	Assembly	90	10
Brownie Sundae	Desserts	119	Assembly	120	30
Soft Serve Cone	Desserts	49	Assembly	60	Not tracked
Apple Pie	Desserts	79	Fry	150	Not tracked
Cookie	Desserts	39	Assembly	30	Not tracked

Choco Lava Cake has a stock of only 10 on purpose: it is the item for the overselling demonstration (AT-04, NFR-CONC-02).

10.5 Modifier groups

Group	Type	Min to max	Options (price change in INR)	Applies to
Spice level	SINGLE	0 to 1	Mild (0), Medium (0), Hot (0)	All chicken items, Spicy Chicken Burger, Peri Peri Fries
Extras	MULTIPLE	0 to 3	Extra cheese (+20), Extra patty (+60), Jalapenos (+15), No onion (0)	All burgers and the wrap
Size	SINGLE	1 to 1	Regular (0), Large (+30)	Cola, Lemon Iced Tea, Cold Coffee, Mango Shake
Dip	MULTIPLE	0 to 2	Cheese dip (+25), Mayo dip (+15)	Veg Nuggets, Hot Wings, Popcorn Chicken

10.6 Performance dataset generator (FR-CFG-03)

- **Parameters:** 180 days ending yesterday, 100,000 orders, a fixed random seed so every run is identical. It refuses to run on a database that already holds orders unless a reset flag is given.
- **Timing:** hour-of-day weights peak at 12 to 14 and 19 to 21; weekends carry 1.3 times the weekday weight. The business day follows the 04:00 cutoff and tokens run in time order within each day.
- **Content:** 1 to 4 lines per order (mean about 2.5), quantity 1 to 3, items weighted so the top 10 account for about 60 percent, modifiers on about 40 percent of lines. Prices, GST and totals use the same formulas as the application, so every CHECK passes.
- **Outcomes:** 97 percent COMPLETED and 3 percent CANCELLED (cancelled in CREATED, with no payment). Payment mix 50 percent UPI, 30 percent CARD, 20 percent CASH.
- **Tickets:** one per distinct station among the order's items (about 2 per order). Cycle time is the target times a random factor between 0.6 and 1.6, so about 12 percent of tickets are late.
- **Audit:** five events per order (created, paid, in kitchen, ready, completed), giving at least 500,000 rows (NFR audit search target).
- **Mechanics:** set-based INSERT ... SELECT with generate_series, ids from the sequences, run as kf_migrator in one transaction, followed by ANALYZE. Target: under 2 minutes on the reference laptop.
- **Checks after loading:** row counts, and the consistency queries C1 to C7 in section 14.2 all return zero rows.

11. Migrations and environments

11.1 Files

```
db/
  ops/roles.sql                roles, schema, session limits (section 4);
run by Compose init
  migration/V1__baseline.sql   everything in section 5
  seed/seed_demo.sql           catalogue, stations, configuration, stock
(section 10)
  seed/gen_orders_100k.sql     performance dataset generator (section 10.6)
  checks/verify_schema.sql     the schema checks of section 11.4
```

11.2 Baseline strategy

While the schema is still changing, V1 is replaced wholesale rather than patched. A developer resets the local database (docker compose down -v) whenever V1 changes, and CI always applies V1 to an empty database. When the first demo build is cut, V1 is frozen and tagged.

11.3 After the first demo

- Changes are forward-only migrations (V2, V3 and so on). An applied migration is never edited; Flyway's checksum validation fails the build if it is.
- Breaking changes use expand and contract: add a nullable column, deploy code that writes both, backfill, add the constraint as NOT VALID and then VALIDATE it, and drop the old column in a later migration.
- Large index builds use CREATE INDEX CONCURRENTLY in a migration marked as non-transactional.
- Rollback means redeploying the previous image and restoring the dump taken before the migration. Every migration pull request carries a one-line rollback note.

11.4 Automated schema checks

Run in CI after the migration; each query must return zero rows.

```
-- 1. No floating-point columns (DD-04)
SELECT table_name, column_name FROM information_schema.columns
WHERE table_schema = 'kf' AND data_type IN ('real', 'double precision');

-- 2. Every table has a primary key
SELECT c.relname FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'kf' AND c.relkind = 'r'
  AND NOT EXISTS (SELECT 1 FROM pg_constraint k WHERE k.conrelid = c.oid AND
k.contype = 'p');

-- 3. Naming convention for foreign keys, unique and check constraints (DD-
03)
SELECT k.conname FROM pg_constraint k
JOIN pg_namespace n ON n.oid = k.connamespace
WHERE n.nspname = 'kf' AND k.contype IN ('f', 'u', 'c')
  AND k.conname !~ '^(fk|uq|ck)_';
```

```
-- 4. Timestamps are timezone-aware (DD-04); the Flyway history table is
excluded
SELECT table_name, column_name FROM information_schema.columns
WHERE table_schema = 'kf' AND data_type = 'timestamp without time zone'
  AND table_name <> 'flyway_schema_history';
```

12. Backup, retention and maintenance

- **Backup:** a nightly pg_dump in custom format, keeping 7 daily copies. A weekly restore drill loads the latest dump into a scratch database and runs the consistency queries (section 14.2) and a row-count comparison.
- **Retention:** orders and audit records are kept for at least 12 months (DR-09). The only purges are idempotency_record rows older than 24 hours and refresh_token rows that expired more than 7 days ago. They run as scheduled jobs in batches of 1,000 rows so transactions stay short, as kf_app, which holds DELETE on exactly those tables.
- **Vacuum and statistics:** autovacuum stays on with the per-table settings in section 5.11. Run ANALYZE after any bulk load. Watch pg_stat_user_tables and investigate when dead tuples exceed 20 percent of a hot table.
- **Statement statistics:** enable pg_stat_statements in the Compose PostgreSQL image and review the slowest statements once during the hardening phase (OI-05).
- **Connections:** PostgreSQL's default of 100 connections is far above the application pool of 20 (SAD 5.4), leaving headroom for the migrator, backups and manual sessions.
- **Time zone:** the server and the application role run in UTC.

13. JPA mapping notes

Entity	Table	Id generation	Notes
Order	orders	IDENTITY	@Version on version; status as an enum stored as a string; lines loaded with a fetch join or entity graph; no cascade remove
OrderLine	order_line	SEQUENCE kf.seq_order_line, allocation size 50	Batch inserts; money as BigDecimal with scale 2
OrderLineModifier, OrderLineComponent	order_line_modifier, order_line_component	SEQUENCE, allocation size 50	Batch inserts
Ticket	ticket	SEQUENCE kf.seq_ticket, allocation size 50	@Version; queue queries list the open statuses explicitly so the partial index applies
TicketItem	ticket_item	SEQUENCE, allocation size 50	Batch inserts
AuditEvent	audit_event	SEQUENCE, allocation size 50	Immutable entity with no setters
All other entities	Their tables	IDENTITY	Low volume

Global settings:

- spring.jpa.hibernate.ddl-auto = validate, so the schema is owned by Flyway (DD-12), and spring.jpa.open-in-view = false.
- hibernate.jdbc.batch_size = 50, order_inserts = true, order_updates = true, and hibernate.jdbc.time_zone = UTC.
- The Hibernate second-level cache is off; the application cache is the only cache (SAD 5.3).
- JSONB columns map with the JSON type annotation. The default snake_case naming strategy matches the DDL.
- Statements S1, S2, S3, S5 and S8 are native queries that return a row count. They bypass the entity version check, so each one sets version and updated_at itself (DD-06).

14. Verification plan

14.1 Checks

Check	Method	Pass condition
Baseline applies	Testcontainers PostgreSQL: run roles.sql, then V1, on an empty database	No errors; the schema checks in 11.4 return zero rows
Constraint behaviour	One test per CHECK, UNIQUE and foreign key: insert a violating row	The expected SQLSTATE: 23514, 23505 or 23503
Role privileges	Connect as kf_app	UPDATE and DELETE on audit_event fail; DDL fails; DELETE on orders fails
Audit trigger	Connect as kf_migrator	UPDATE and DELETE on audit_event raise the exception
Concurrency	The SAD 6.6 tests (NFR-CONC-01 to 04)	Pass 50 repeats; the mutation check turns them red
Plan targets	100,000-order dataset, EXPLAIN (FORMAT JSON)	Plans meet section 7.2

Check	Method	Pass condition
Report ground truth	Compare S10 to S13 with an independent Java calculation on a small known dataset	Identical figures
Generator	Run on an empty database	Row counts as specified; C1 to C7 return zero rows; under 2 minutes
Seed	Run on an empty database	Under 60 seconds (FR-CFG-02); every item has a station (BR-09)

14.2 Consistency queries

Each query returns zero rows on a healthy database. They run after the generator, in the weekly restore drill, and as a scheduled check in the polish phase.

```
-- C1 Order totals equal the sum of their lines
SELECT o.id FROM kf.orders o
JOIN kf.order_line l ON l.order_id = o.id
GROUP BY o.id, o.subtotal, o.tax_total
HAVING o.subtotal <> sum(l.line_subtotal) OR o.tax_total <> sum(l.line_tax);

-- C2 Every order past CREATED has a payment
SELECT o.id FROM kf.orders o
LEFT JOIN kf.payment p ON p.order_id = o.id
WHERE o.status IN ('PAID','IN_KITCHEN','READY','COMPLETED') AND p.id IS NULL;

-- C3 A READY or COMPLETED order has no open ticket
SELECT o.id FROM kf.orders o
JOIN kf.ticket t ON t.order_id = o.id
WHERE o.status IN ('READY','COMPLETED') AND t.status IN
('NEW','IN_PROGRESS');

-- C4 An IN_KITCHEN order has at least one ticket
SELECT o.id FROM kf.orders o
WHERE o.status = 'IN_KITCHEN'
  AND NOT EXISTS (SELECT 1 FROM kf.ticket t WHERE t.order_id = o.id);

-- C5 Tickets mirror their order's rush flag and business day (DD-09)
SELECT t.id FROM kf.ticket t JOIN kf.orders o ON o.id = t.order_id
WHERE t.rush <> o.rush OR t.business_day <> o.business_day;
```

```
-- C6 Every menu item has at least one station (BR-09)
SELECT i.id FROM kf.menu_item i
WHERE NOT EXISTS (SELECT 1 FROM kf.item_station s WHERE s.item_id = i.id);

-- C7 Tokens have no gaps within a business day (a rolled-back order must not
consume one)
SELECT business_day FROM kf.orders
GROUP BY business_day
HAVING max(token) <> count(*);
```

15. Traceability and proposed changes

15.1 Traceability

Requirement	Addressed in
DR-01, DR-02 (money, UTC)	DD-04; schema checks in 11.4
DR-03 (foreign keys and checks)	Section 5; tests in 14.1
DR-04 (unique constraints)	uq_ constraints in 5.7 to 5.9
DR-05 (no deletion of referenced rows)	DD-08; grants in 5.11
DR-06 (snapshots)	order_line and ticket_item in 5.7 and 5.9
DR-07 (indexes)	Sections 7.1 and 7.2
DR-08 (volume basis)	Sections 9 and 10.6
DR-09 (retention)	Sections 9 and 12; grants in 5.11
BR-01 to BR-18	Section 6
FR-CFG-02, FR-CFG-03	Section 10
FR-AUD-02	Sections 5.10 and 5.11
NFR-PERF-01, 03, 04	Sections 7.2 and 8
NFR-CONC-01 to 04	Sections 6 and 8 (S1 to S8); 14.1
NFR-REL-03	Section 11

15.2 Proposed changes to the SRS and SAD

1. SAD 4.1: station is owned by the config module.
2. SRS section 9: add the tables menu_version and order_line_component, and the columns ticket.business_day, ticket.rush, ticket.cancel_ack_at, ticket.recall_count, orders.cancel_reason_code, orders.cancel_reason_text, orders.cancelled_by and payment.change_due.
3. SRS section 9, IdempotencyRecord: remove the status attribute, because the row is invisible until it is complete.
4. SRS DR-07: replace the generic index list with the partial indexes in section 7.1.
5. SAD 5.5: state that batch inserts need sequence-based ids with INCREMENT 50, not identity columns (DD-02).
6. SAD 5.4: the report statement timeout of 10 seconds is set per transaction with SET LOCAL; the role default stays at 5 seconds.
7. SRS FR-PAY-06: state that change_due is stored with the payment.

16. Open items and next step

16.1 Open items

ID	Item	When
OI-01	Pin the PostgreSQL major version	Phase 0
OI-02	Confirm the combo expansion snapshot (order_line_component)	Document 6
OI-03	Enforce BR-09 with a deferrable constraint trigger, or keep it in the service and a test	Phase 3
OI-04	Partition audit_event by month if retention or volume grows beyond the design basis	When needed
OI-05	Enable pg_stat_statements in the Compose image	Phase 3
OI-06	Decide whether GST should be per item instead of per category; it is per category today	Document 5

16.2 Next step

Document 5, the API Specification: every endpoint with request and response schemas, error examples, the role and rate-limit class of each endpoint, and the OpenAPI definition generated from the code. It builds on the SRS section 5 requirements and the stream and error designs in SAD sections 9 and 5.

Version	Date	Author	Change
0.1	29 October 2026	Rathi	First draft